PYTHON · МОЙ ПИТОМЕЦ / ЧАСТЬ 2

Превращаем свою игру в тамагочи

Я приготовил для тебя окно на Pygame. В нём уже есть кнопки, картинка питомца и полоски состояния. Ты будешь управлять этим окном несколькими командами, а характер и правила игры останутся твоими.

1. Подготовь папку

Я заранее помогу установить Pygame на компьютере. Скопируй в свою папку проекта файл `pet_window.py` и папку `images`. Они должны лежать рядом с твоим `main.py`. Запускай именно `main.py`.

Перед изменениями сохрани текстовую версию под именем `text_game.py`: в Проводнике VS Code скопируй свой файл, вставь в ту же папку и переименуй копию. В этой инструкции мы продолжаем работать в `main.py`.

Если начинаешь с моего готового графического примера, нужные изменения уже внесены. Прочитай следующие шаги, чтобы понять их, затем переходи к своему герою и проверке. Не добавляй одинаковые команды второй раз.

2. Подключи окно

Самой первой строкой напиши:

```
from pet_window import start, show, choose, say, finish
```

Это означает: «возьми пять готовых команд из файла pet_window.py». Их можно вызывать так же, как знакомые `print()` и `input()`. Устройство этих команд мы пока не пишем.

3. Открой окно и познакомь нас с героем

Вместо прежних приветствий и `name = input(...)` напиши:

```
name = "Пушок"
start(name, "Кот с Сатурна")
say("Привет! Я", name, "и я обожаю звёздное печенье.")
```

Теперь имя задаёшь ты, в программе. Замени его, вид питомца и приветствие. Игра общается в окне; отвечать ей в терминале больше не нужно.

4. Замени текстовые показатели графическими

Переменные `hunger`, `energy`, `mood`, `day`, `total_days` оставь как были. Внутри большого цикла удали команды `print()`, которые показывали день, показатели и список действий. На их месте напиши:

```
    show(hunger, energy, mood, day, total_days)
    choice = choose("Покормить", "Поиграть", "Спать")
```

У обеих строк четыре пробела слева. `show()` обновляет экран, `choose()` ждёт нажатия одной из трёх кнопок и записывает её номер в `choice`. Порядок значений в `show()` важен: голод, энергия, настроение, день, всего дней.

Удали прежний `choice = input(...)` и маленький цикл проверки неправильного ввода. У кнопок уже есть только три допустимых ответа.

Большой цикл `while day <= total_days:` оставь: он по-прежнему считает дни.

5. Перенеси реплики в окно

В действиях и реакциях замени `print(...)` на `say(...)`. Например:

```
    if choice == "1":
        hunger = hunger - 3
        say(name, "съел целую тарелку звёзд!")
```

Не меняй вычисления, ограничения от 0 до 10 и `day = day + 1`. Удали старую строку `print("К вечеру: ...")`: показатели уже видны на экране. Результат действия появится перед следующим выбором. После нажатия кнопки предыдущие реплики уступят место новым.

Важная деталь: окно показывает **сытость**, а программа хранит **голод**. Чем меньше голод, тем полнее зелёная полоска. Это специально сделано так, чтобы полная полоска всегда означала хорошее состояние.

6. Покажи окончание игры

После большого цикла, без отступа слева, добавь обновление последнего дня:

```
show(hunger, energy, mood, total_days, total_days)
```

Старые команды вывода итогов замени на выбор звания и `finish()`:

```
if mood >= 7:
    title = "Лучший друг звёздного кота!"
elif energy >= 7:
    title = "Хранитель космических снов!"
else:
    title = "Исследователь кошачьей планеты!"

finish(title, "Спасибо за эти десять дней!")
```

`title` хранит выбранное звание. `finish()` показывает его в окне и ждёт, когда игрок нажмёт «Завершить». Эта команда должна быть последней в игре. Не добавляй `input()` для паузы: окно уже умеет ждать само.

7. Нарисуй своего питомца

Без файлов ты увидишь моего лиса. Чтобы поселить своего героя:

1. Открой Paint и нарисуй питомца, например на картинке 256 × 256 пикселей.
2. Выбери **Сохранить как → Изображение PNG**.
3. Открой свою папку проекта, затем папку `images`.
4. Сохрани рисунок с именем `pet.png`.
5. Останови игру и запусти её снова.

Картинка автоматически уменьшится или увеличится, сохраняя пропорции. Можно подготовить разные выражения лица и сохранить их рядом: `happy.png` — радость, `hungry.png` — голод, `tired.png` — усталость, `sleeping.png` — сон. Достаточно одной картинки, остальные — по желанию.

8. Добавь свою особенность

Я предлагаю выбрать одну идею:

- переименуй действия: «Зарядить», «Танцевать», «Спящий режим»;
- добавь свои реплики на разные состояния;
- сохрани особое событие пятого дня из текстовой игры;
- нарисуй смешную реакцию на сон;
- поменяй цвет акцента: `start(name, "Кот с Сатурна", color="#247b73")`.

Названия, реплики и правила можно менять независимо от устройства окна. При желании отключи покачивание: добавь `motion=False` в `start()`. То же самое делает клавиша M во время игры.

9. Проверь и покажи

Нажми Ctrl + S. Запусти `main.py` кнопкой **Run Python File in Terminal**. Если появилось `No module named pygame`, позови меня: нужно выбрать Python, в котором мы установили Pygame. Сам код игры из-за этого менять не нужно.

Проверь:

- □ Открылось окно с именем моего питомца.
- □ Каждая кнопка меняет нужные показатели и расходует один день.
- □ С клавиатуры работают цифры 1, 2, 3.
- □ Мои реплики читаются; длинные можно прокручивать колёсиком или ↑/↓.
- □ После десятого действия видны итоговые показатели и звание.
- □ Крестик закрывает игру даже во время ожидания выбора.
- □ Если я нарисовал PNG, игра показывает именно моего героя.

Теперь пригласи соседа поиграть. Объясни ему одну особенность своего питомца, которой нет в моём примере.